BIKAJI Remediation — Final Status Assessment

System: **ELITE_BREAKOUT_SYSTEM** | Target Branch: `main` | Governance Review

DEPLOYMENT: BLOCKED | D4: REPORTED FIXED | D1: EVIDENCE NEEDED | D6: TESTS REQUIRED | POSTGRESQL: PENDING

> **Executive Overview:** Meaningful progress has been achieved across the remediation suite. The report documents that all 27 selected tests pass (including 12 new freshness/lifecycle tests, 11 BIKAJI regression tests, and 4 scanner-isolation tests). However, because no real PostgreSQL concurrency test was executed, the deployment block is correctly retained.
>
> *Note: This assessment is based on the provided status report and pre-deployment verification criteria.*

1. Review of the Three Reported Fixes

D4 — Historical Indicator Freshness: Promising, Subject to Code Verification

The fix distinguishes the live CMP update from historical indicator freshness. Live-price injection no longer resets `row["is_stale"]`, and the complete pipeline is tested:

```
calculate_wealth_technicals() → live CMP injection → evaluate_open_positions() → DATA_STALE
```

This addresses the weakness in the prior audit where prebuilt stale rows bypassed the live-price injection path. **Acceptance Condition:** The integration test must prove that a fresh CMP cannot make stale SMA200 or RS data actionable, the alert remains `OPEN`, and no terminal closure occurs, while verified fresh data still executes valid exits.

Assessment: PROVISIONALLY ACCEPTED (source-level certification remains outstanding).

D1 — False Rejection After PostgreSQL Commit: Not Yet Fully Certified

The updated behavior returns: `True, "INSERTED_NEW_ALERT_LIFECYCLE_SYNC_PENDING_<alert_id>"` when PostgreSQL has persisted the BUY but separate SQLite lifecycle registration fails. While superior to reporting a false failure, returning success does not by itself reconcile the lifecycle gate. Three requirements must be demonstrated:

1. **Return-contract compatibility:** Audit all callers of `save_v2_candidate_alert()` to confirm boolean/message tuples are handled consistently without silent skips.
2. **Durable recovery:** Show where synchronization-pending events are persisted, how they survive process restarts, how they are retried, and how eligibility checks treat unresolved lifecycle state.
3. **Both failure directions:** Test PostgreSQL commit failure after SQLite operation succeeds, as well as SQLite failure after PostgreSQL commits.

Assessment: IMPROVED SEMANTICS — NOT FULLY CERTIFIED (reconciliation mechanism and caller contracts pending proof).

D6 — Terminal UPSERT Protection: Right Direction, Test Is Insufficient

The conditional update clause added to PostgreSQL:

```
ON CONFLICT (symbol, breakout_type, scanner, alert_date)
DO UPDATE
SET ...
WHERE alerts.status IN ('OPEN', 'ACTIVE')
RETURNING id, (xmax = 0) AS is_insert
```

When an existing row is terminal (`WIN`, `LOSS`, `CLOSED`), the `WHERE` condition evaluates to false, causing PostgreSQL to return **no row** (`fetchone() is None`).

Required Behavioral Checks:

- An `OPEN` conflict updates intended fields without creating duplicate alerts.
- A `WIN`, `LOSS`, or `CLOSED` conflict remains terminal.
- Existing exit price, P&L, exit reason, and lifecycle fields remain unmodified.
- The no-row result (`None`) is safely handled by application code.
- Repeated scans cannot generate another eligible BUY through a caller-side fallback.

Assessment: PROMISING SQL CLAUSE — NOT BEHAVIORALLY CERTIFIED.

2. Remaining Acceptance Gates

Requirement	Current Assessment
D4 — Stale indicators cannot be cleared by live CMP	Reported fixed; verify source and full-pipeline integration test.
D1 — No false rejection after committed BUY	Improved; durable reconciliation and caller contracts require proof.
D6 — Terminal UPSERT protection	Clause added; behavioral execution verification required.
D2 — Lifecycle synchronization after closure	No completed fix demonstrated in current report.
D3 — SELL_REVIEW lifecycle synchronization	No completed fix demonstrated in current report.
D5 — Manual-position lot deletion	No completed fix demonstrated in current report.
Concurrent closure and re-entry	**BLOCKED** pending genuine PostgreSQL test execution.
Official closing-price provenance	Still requires evidence of actual authoritative price used.
No live broker orders	Reported preserved; confirmed zero live broker dispatch.

3. Exact Next Instructions for the Coding Agent

Do not start another broad redesign. Complete the evidence gaps against the current working tree:

1. **Audit D1**
   Enumerate every caller of `save_v2_candidate_alert()`. Prove return values are handled consistently. Identify durable source of lifecycle synchronization-pending work. Demonstrate retry, idempotency, process-restart recovery, and eligibility behavior while synchronization is pending. Test both PostgreSQL-commit failure and SQLite-registration failure.

2. **Audit D4**
   Review the full stale-indicator implementation and integration test. Confirm live CMP updates only CMP freshness and never clears indicator freshness. Assert final persisted position status and verify no downstream path converts `DATA_STALE` into terminal closure.

3. **Audit D6 Behaviorally**
   Execute UPSERT against an actual test PostgreSQL database. Test active and terminal conflicts, inspect returned rows and affected fields, and prove no terminal state or exit data can be overwritten. Audit all handling of `fetchone() is None`.

4. **Close D2 and D3**
   Implement durable lifecycle synchronization for terminal closure and `SELL_REVIEW` transitions. Ensure retries cannot apply older events after newer terminal events.

5. **Resolve D5**
   Verify whether multiple manual lots are supported. Preserve independent positions if supported; otherwise demonstrate enforced single-position invariant.

6. **Run PostgreSQL Staging Tests**
   Use independent connections and concurrent workers to test same-day BUY races, competing closure attempts, symbol/alert-ID mismatch, terminal UPSERT protection, and lifecycle-event recovery. Report tests as **BLOCKED** if no isolated PostgreSQL instance can be obtained.

7. **Verify Price Provenance**
   Establish actual source, trading date, and validation rules for 18:30 exit price. Do not describe LTP as official exchange close without supporting evidence.

8. **Full Regression, Syntax, & Diff Review**
   Run the full relevant regression suite, syntax checks, and a complete Git-diff review. Investigate prior cross-strategy test failures rather than assuming they are pre-existing.

Final Governance Verdict

DO NOT PROMOTE THIS BUILD YET. The 27 passing tests are encouraging, and D4 appears to have received a more meaningful integration test. But D1 must demonstrate durable reconciliation rather than merely return a success indicator, D6 needs an executed behavioral test, and the real PostgreSQL concurrency requirement remains unfulfilled.

The next certification report should distinguish clearly between fixes implemented, fixes independently verified, and tests that remain blocked.